

Uma Metaheurística VNS para o Problema de Alinhamento Sequência-Grafo com Modificação de Rótulos

RESUMO

Este trabalho aborda o Problema de Alinhamento Sequência-Grafo com Modificação de Rótulos, um problema NP-difícil em que o objetivo consiste em encontrar um passeio em um grafo rotulado que reproduza uma sequência de consulta, minimizando as modificações necessárias nos rótulos originais dos vértices. Para isso, apresentamos uma formulação de Programação Linear Inteira e propomos uma metaheurística de Busca em Vizinhaça Variável (VNS). Experimentos computacionais mostraram que o modelo exato resolve instâncias menores até a otimalidade, mas falha em grafos esparsos com consultas longas. Nesses cenários intratáveis, a VNS demonstrou alta robustez e competitividade, encontrando alinhamentos perfeitos e superando o melhor limitante superior da abordagem exata em mais de 35% nas instâncias mais complexas, consolidando-se como uma alternativa eficaz.

PALAVRAS CHAVE. Alinhamento Sequência-Grafo, Busca em Vizinhaça Variável (VNS), Programação Linear Inteira, Otimização Combinatória.

Tópicos (Metaheurísticas, Programação Matemática, Otimização Combinatória)

ABSTRACT

This paper addresses the Sequence-to-Graph Alignment Problem with Vertex Label Modification, an NP-hard where the objective is to find a walk in a labeled graph that matches a query sequence while minimizing the required alterations to the original vertex labels. To this end, we show an Integer Linear Programming (ILP) formulation and propose a Variable Neighborhood Search (VNS) metaheuristic. Computational experiments revealed that the exact model solves smaller instances to optimality but fails on sparse graphs with long queries. In these intractable scenarios, the VNS demonstrated high robustness and competitiveness, finding perfect alignments and outperforming the exact approach's best upper bound by over 35% in the hardest instances, establishing itself as a highly effective alternative.

KEYWORDS. Sequence-to-Graph Alignment, Variable Neighborhood Search (VNS), Integer Linear Programming, Combinatorial Optimization.

Paper topics (Metaheuristics, Mathematical Programming, Combinatorial Optimization)

1. Introdução

Nos últimos anos, a pesquisa operacional vem ganhando mais espaço no mundo acadêmico e industrial, consolidando-se como um pilar em áreas como a Inteligência Artificial (IA) e a Biologia Computacional. Na IA, a PO é essencial tanto no treinamento de Grandes Modelos de Linguagem (LLMs) quanto na mitigação de suas limitações. Dado que as LLMs permeiam o dia a dia do ser humano contemporâneo, é primordial melhorar a corretude das respostas, diminuindo as ditas *alucinações*. Com essa motivação, Sansford et al. [2024] desenvolveram o GraphEval, que modelam o output da LLM como um grafo de referência, e assim, podem alimentar uma sequência correta nessa referência para corrigir uma alucinação. Já na biologia computacional, a PO surge por exemplo no polishing de montagens genômicas [Vaser et al., 2017], em que leituras confiáveis são usadas para corrigir erros nos rótulos de um grafo de montagem construído a partir de leituras ruidosas, recuperando-se assim a sequência original. Em ambos os cenários, a capacidade de alinhar sequências arbitrárias a estruturas de grafos é fundamental para a extração e representação precisa de informações, problema este denominado de *Alinhamento Sequência-Grafo*.

Mais formalmente, o problema de *Alinhamento Sequência-Grafo* considera um grafo dirigido rotulado $G = (V, E, \sigma)$, onde $\sigma : V \rightarrow \Sigma^+$ atribui a cada vértice $v \in V$ uma *string* sobre o alfabeto Σ , e uma sequência de consulta $s \in \Sigma^m$ de comprimento m . Um passeio $\pi = \langle v_0, v_1, \dots, v_{m-1} \rangle$ em G induz a sequência $\sigma(v_0) \sigma(v_1) \dots \sigma(v_{m-1})$. Na versão clássica do problema, busca-se minimizar a distância de edição entre s e a sequência induzida por π . Na variante estudada neste trabalho, os rótulos dos vértices podem ser alterados; o objetivo é então encontrar um passeio π e um conjunto mínimo de modificações $\mathcal{M}$ nos rótulos de G tal que π passe a induzir exatamente s . A Figura 1 apresenta um exemplo do Problema de Alinhamento Sequência-Grafo com Modificação de Rótulos.

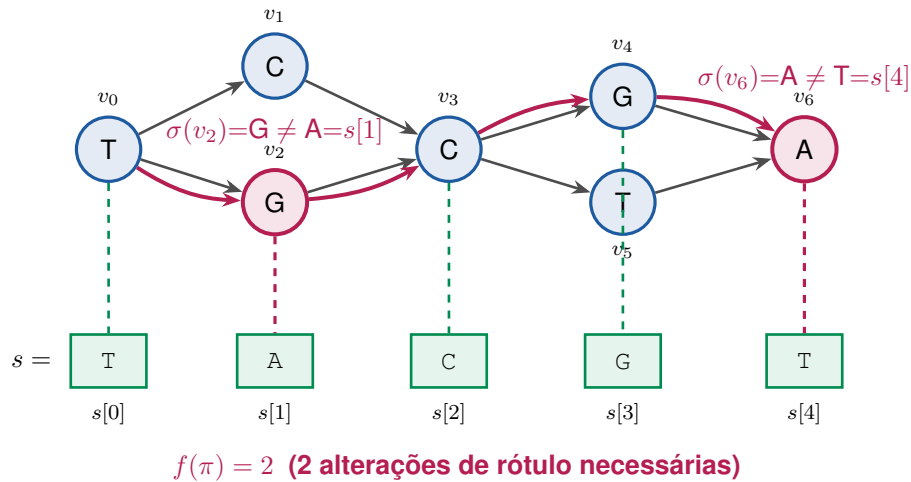


Figura 1: Exemplo de uma instância do Problema de Alinhamento Sequência-Grafo com Modificação de Rótulos. O passeio destacado $\pi = \langle v_0, v_2, v_3, v_4, v_6 \rangle$ induz a sequência TGCGA, que difere da consulta $s = TACGT$ nas posições 1 e 4, exigindo duas modificações de rótulo.

Como demonstrado por Jain et al. [2020], este problema é $\mathcal{NP}$ -completo para $|\Sigma| \geq 2$. Apesar da relevância do problema, até onde sabemos, na literatura não foram propostos algoritmos exatos ou heurísticos para a variante estudada nesse problema. Com isso, neste trabalho, apresentamos uma metaheurística Variable Neighbourhood Search (VNS) e uma formulação exata de uma programação linear inteira (PLI) para o problema.

Este artigo está estruturado da seguinte forma: a Seção 2 aborda a revisão bibliográfica e os trabalhos relacionados. A Seção 3 apresenta a formulação de Programação Linear Inteira (PLI) para o problema. Em seguida, a Seção 4 detalha a metaheurística VNS proposta. Por fim, os experimentos computacionais são discutidos na Seção 5, seguidos pelas conclusões e sugestões para trabalhos futuros na Seção 6.

2. Revisão Bibliográfica

Em Amir et al. [2000], os autores provam que o Alinhamento de Sequência em Grafos com Modificação de Rótulos é $\mathcal{NP}$ -difícil para $|\Sigma| \geq |V|$ a partir da demonstração da assimetria dos locais de edição, permitindo a redução do problema para o do Caminho e Ciclo Hamiltoniano Direcionado, assumindo alterações no grafo ou simultaneamente no grafo e na sequência.

Em Jain et al. [2020], em contraste com Amir et al. [2000], cujas provas de NP-completude para o alinhamento com modificações no grafo dependiam da premissa de um alfabeto grande o suficiente para rotular cada vértice de forma distinta ($|\Sigma| \geq |V|$), Jain et al. [2020] demonstraram que o problema permanece $\mathcal{NP}$ -difícil mesmo para alfabetos de tamanho constante ($|\Sigma| \geq 2$). Em vez de depender de caracteres exclusivos por vértice, eles estruturaram a prova utilizando *tokens* de sequências binárias repetitivas na consulta, provando que a intratabilidade do problema deriva da natureza cíclica dos grafos, e não do tamanho do alfabeto.

Apesar de na literatura não terem trabalhos que tratam da variante específica desse trabalho, soluções exatas para versão clássica do problema do Alinhamento de Sequências em Grafos já foram estudadas por diversos autores, como GraphAligner [Rautiainen e Marschall, 2020], AStarix [Ivanov et al., 2020], Theseus [Jiménez-Blanco et al., 2026], [Jain et al., 2019], entre outros. Porém, como são soluções exatas para um problema $\mathcal{NP}$ -difícil, são limitados a alinhamentos com grafos e sequências menores.

3. Formulação de Programação Linear Inteira

Nesta seção apresenta-se uma formulação de PLI para o problema. Dado um grafo orientado $G = (V, E, \sigma)$, em que cada vértice $v \in V$ carrega um rótulo original $\sigma(v)$ pertencente a um dado alfabeto Σ . Considere uma sequência de consulta q de comprimento m , indexada como $q[1] \dots q[m]$. O problema consiste em encontrar um passeio em G que reproduza a exata sequência q minimizando a quantidade de vértices cujos rótulos originais precisem ser modificados.

Para estruturar esta formulação, introduzimos os seguintes elementos:

- $\delta^-(v) = \{u \in V : (u, v) \in E\}$: denota o conjunto de vértices predecessores ao vértice v ;
- $\mathcal{A} = \{(i, j) : 1 \leq i < j \leq m, q[i] \neq q[j]\}$: define o conjunto de pares de posições na sequência de consulta que demandam caracteres incompatíveis.

O espaço de soluções é mapeado por duas variáveis de decisão binárias. A variável $y_{v,i} \in \{0, 1\}$ indica se o vértice $v \in V$ é alocado para a i -ésima posição do alinhamento ($y_{v,i} = 1$) ou não ($y_{v,i} = 0$). Ademais, a variável $S_v \in \{0, 1\}$ atua como uma penalidade, assumindo o valor 1 caso o rótulo original do vértice v sofra alteração, e 0 em caso contrário. O problema então pode ser formulado da seguinte maneira:

$$\text{Minimizar } \sum_{v \in V} S_v \quad (1)$$

$$\text{sujeito a } \sum_{v \in V} y_{v,i} = 1, \quad \forall i \in \{1, \dots, m\} \quad (2)$$

$$y_{v,i+1} \leq \sum_{u \in \delta^-(v)} y_{u,i}, \quad \forall v \in V, i \in \{1, \dots, m-1\} \quad (3)$$

$$y_{v,i} + y_{v,j} \leq 1, \quad \forall v \in V, (i, j) \in \mathcal{A} \quad (4)$$

$$S_v \geq y_{v,i}, \quad \forall v \in V, i \in \{1, \dots, m\} \text{ tal que } q[i] \neq \sigma(v) \quad (5)$$

$$y_{v,i}, S_v \in \{0, 1\}, \quad \forall v \in V, i \in \{1, \dots, m\} \quad (6)$$

A função objetivo (1) visa minimizar o total de vértices que terão seus rótulos originais alterados. Para garantir a viabilidade do passeio, as restrições (2) impõem que cada passo i da sequência de consulta seja coberto por um e apenas um vértice do grafo. A fim de que a sequência de vértices selecionados forme de fato um caminho válido, as restrições de fluxo (3) exigem coerência topológica: a seleção de um vértice v para o passo $i + 1$ está estritamente condicionada à visita prévia de ao menos um de seus predecessores $u \in \delta^-(v)$ no passo imediatamente anterior i .

Um ponto importante da modelagem é a preservação da integridade estrutural do grafo, que dita que cada vértice deve possuir apenas um rótulo final. As restrições (4) asseguram essa propriedade ao proibir que um mesmo vértice v represente posições da consulta que exijam caracteres distintos (pares no conjunto $\mathcal{A}$).

Ademais, as restrições (5) forçam a variável S_v a assumir o valor 1 sempre que um vértice v for selecionado para uma posição i em que o caractere exigido $q[i]$ divergir do rótulo original $\sigma(v)$. Por fim, (6) restringe o domínio de todas as variáveis ao conjunto binário.

4. Metaheurística de Busca em Vizinhança Variável

Considerando que a complexidade de pior caso dos métodos exatos cresce de forma proibitiva com o tamanho da instância, propomos nesta seção uma metaheurística baseada na *Busca em Vizinhança Variável* (Variable Neighborhood Search — VNS) [Hansen e Mladenović, 2001; Hansen et al., 2010]. O VNS opera diretamente sobre o espaço combinatório de soluções, de modo que cada candidato é um passeio $W = \langle v_0, v_1, \dots, v_{m-1} \rangle$ de comprimento m no grafo rotulado $G(V, E, \sigma)$, sujeito à restrição de viabilidade de arco em cada etapa.

O princípio central do VNS é a exploração sistemática de múltiplas estruturas de vizinhança hierarquicamente ordenadas, $\mathcal{N}_1, \mathcal{N}_2, \dots, \mathcal{N}_{K_{\max}}$, cada qual definindo uma noção distinta de proximidade no espaço de soluções. A motivação é que um ótimo local sob uma estrutura de vizinhança raramente o é sob uma estrutura diferente [Hansen e Mladenović, 2001]: ao escapar sequencialmente de ótimos locais por meio de perturbações progressivamente maiores e ao aplicar busca local imediatamente após cada perturbação, o algoritmo alterna entre exploração e aproveitamento de forma sistemática. A variante implementada é a *Busca em Vizinhança Variável Geral* (General Variable Neighborhood Search — GVNS) [Hansen et al., 2010], na qual a fase de busca local é executada até que nenhum movimento de melhoria exista sob a primeira estrutura de vizinhança, garantindo que toda solução aceita seja um ótimo local antes que o próximo ciclo de perturbação se inicie. A mecânica completa do algoritmo proposto é apresentada no Algoritmo 1.

Uma solução é representada como um passeio $W = \langle v_0, v_1, \dots, v_{m-1} \rangle$ em G , em que cada par consecutivo $(v_{i-1}, v_i) \in E$ para $i \in \{1, \dots, m-1\}$. O custo de um passeio é o número

de posições nas quais o rótulo do vértice visitado diverge do caractere requerido pela sequência de consulta:

$$f(W) = \sum_{i=0}^{m-1} \mathbf{1}[\sigma(v_i) \neq q[i]] + P_{\text{dead}} \cdot (m - |W|),$$

onde $|W|$ denota o comprimento efetivo do passeio quando ele é interrompido prematuramente por um vértice sem arcos de saída (*beco sem saída*), e $P_{\text{dead}} = m + 1$ é uma penalidade suficientemente grande para garantir que qualquer passeio completo seja preferível a qualquer passeio incompleto.

4.1. Construção Gulosa Randomizada

A solução inicial é construída por um procedimento guloso randomizado que explora a compatibilidade de rótulos em cada etapa. Para o vértice inicial, o construtor filtra todos os vértices $v \in V$ cujo rótulo coincide com $q[0]$ e seleciona um deles aleatoriamente; na ausência de tal vértice, qualquer vértice é escolhido de forma uniforme. Em cada posição subsequente $i \in \{1, \dots, m-1\}$, o construtor examina os vizinhos de saída do vértice atual e os particiona em *candidatos favoráveis*, aqueles cujo rótulo coincide com $q[i]$, e *candidatos neutros*, os demais vizinhos. Se ao menos um candidato favorável existir, um deles é selecionado aleatoriamente; caso contrário, um candidato neutro aleatoriamente é escolhido, incorrendo em uma alteração de rótulo. Se o vértice atual não possuir arcos de saída, o passeio é encerrado prematuramente e a penalidade de beco sem saída é aplicada.

O mesmo procedimento de extensão gulosa, denominado ESTENDERGULOSO, é reutilizado ao longo do algoritmo para completar passeios parcialmente construídos após movimentos de perturbação.

4.2. Estruturas de Vizinhança

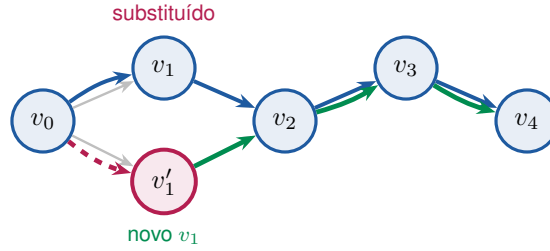
Definem-se quatro estruturas de vizinhança sobre o espaço de passeios, ordenadas por raio de perturbação crescente. Cada estrutura $\mathcal{N}_k$ é caracterizada por um operador de *agitação* (*shake*) que amostra aleatoriamente um vizinho da solução corrente para fins de fuga de ótimos locais. Todos os operadores de agitação preservam a viabilidade do passeio invocando ESTENDERGULOSO para reparar o sufixo posterior ao ponto de modificação.

$\mathcal{N}_1$ — **Substituição de arco simples.** Uma posição i é sorteada uniformemente de $\{1, \dots, |W| - 1\}$. O vértice v_i é substituído por um vizinho de v_{i-1} escolhido aleatoriamente, com preferência por vizinhos distintos de v_i quando existirem. O sufixo $v_{i+1}, \dots, v_{m-1}$ é descartado e reconstruído por ESTENDERGULOSO a partir do novo v_i .

$\mathcal{N}_2$ — **Substituição de arco dupla.** Dois pontos $i < j$ são sorteados uniformemente de $\{1, \dots, |W| - 1\}$. Um novo vértice é amostrado dentre os vizinhos de saída de v_{i-1} , e o segmento $v_i, \dots, v_j$ é reconstruído gulosamente entre as posições i e j . O sufixo $v_{j+1}, \dots, v_{m-1}$ é então completado por ESTENDERGULOSO.

$\mathcal{N}_3$ — **Reinserção de segmento.** Uma posição i e um comprimento de segmento L sorteado uniformemente de $\{2, \dots, \lfloor m/3 \rfloor\}$ são selecionados. O bloco $v_i, \dots, v_{i+L-1}$ é removido e o passeio é reconstruído gulosamente a partir da posição i , iniciando a partir de um vizinho de saída aleatório de v_{i-1} (caso $i > 0$) ou de um vértice inicial recém-sorteado (caso $i = 0$).

$\mathcal{N}_4$ — **Substituições simultâneas múltiplas de arco.** Um conjunto de $k_{\text{pert}} = \min(k, |W| - 1)$ posições distintas é amostrado sem reposição de $\{1, \dots, |W| - 1\}$ e embaralhado. Para cada posição selecionada i nessa ordem, o vértice v_i é substituído por um vizinho de saída aleatório de v_{i-1} , e o sufixo é imediatamente reparado por ESTENDERGULOSO antes que a posição seguinte seja processada. O número de perturbações simultâneas cresce com o índice de vizinhança k , tornando $\mathcal{N}_4$ progressivamente mais agressiva à medida que a busca avança por vizinhanças de maior índice ($k \geq 4$).



$\mathcal{N}_1$: substituição de arco simples

Figura 2: Ilustração do mecanismo de agitação $\mathcal{N}_1$. O mecanismo substitui a decisão de arco na posição 1, trocando v_1 por v'_1 (arco rosa tracejado) e reparando o sufixo por ESTENDERGULOSO (arcos verdes).

4.3. Busca Local: Descida de Primeiro Aprimoramento

A fase de busca local aplica uma descida de *primeiro aprimoramento* restrita a substituições de arco único com compatibilidade de rótulos. A cada iteração da descida, as posições $i = 0, 1, \dots, |W| - 1$ são varridas em ordem. Para $i = 0$, o conjunto de candidatos é composto por todos os vértices $v \in V \setminus \{v_0\}$ com $\sigma(v) = q[0]$; para $i > 0$, é composto pelos vizinhos de saída de v_{i-1} em G que diferem do v_i atual e cujo rótulo coincide com $q[i]$. Para cada candidato w , um passeio de teste é construído retendo o prefixo $v_0, \dots, v_{i-1}$, acrescentando w e completando com ESTENDERGULOSO. Caso o custo resultante melhore a solução corrente, o movimento é aceito imediatamente, a varredura reinicia a partir de $i = 0$ e o processo continua. A descida termina quando uma varredura completa não produz nenhum candidato de melhoria, certificando um ótimo local sob a vizinhança restrita de arco único com compatibilidade de rótulo.

A busca global segue o esquema GVNS [Hansen et al., 2010], no qual o laço principal alterna entre as fases de agitação e busca local sob uma sequência ordenada de vizinhanças. No início de cada iteração externa, o índice de vizinhança é definido como $k = 1$. O operador de agitação amostra um vizinho aleatório W' da solução corrente sob $\mathcal{N}_k$, e a busca local é aplicada a W' para obter uma solução refinada W'' . Se $f(W'') < f(W)$, a solução corrente é atualizada para W'' e o índice de vizinhança é redefinido para $k = 1$; caso contrário, o índice é avançado para $k + 1$. Quando k supera K_{max} , o ciclo interno termina e o contador de iterações externas é incrementado.

Para combater estagnação de longo prazo, um mecanismo de reinício é acionado sempre que a melhor solução global não tiver melhorado por N_{stag} iterações externas consecutivas. Nesse momento, uma nova solução de partida é gerada invocando o construtor guloso randomizado e aplicando busca local ao resultado, de modo que a busca é realocada a uma região diferente do espaço de soluções enquanto a melhor solução global é preservada como referência.

O Algoritmo 1 apresenta o procedimento GVNS completo. O Algoritmo 2 detalha a sub-rotina de busca local por primeiro aprimoramento, e o Algoritmo 3 especifica os quatro operadores de agitação.

Algoritmo 1: Procedimento GVNS

Entrada: Grafo $G(V, E, \sigma)$, sequência de consulta q de comprimento m ,
 iterações máximas $I_{\max}$, índice máximo de vizinhança $K_{\max}$,
 limite de estagnação N_{stag} , penalidade de beco $P_{\text{dead}} = m + 1$.
Saída: Custo mínimo de alteração de rótulo f^* e passeio correspondente W^* .

```

1  $W_0 \leftarrow \text{CONSTRUIRGULOSO}(G, q)$ 
2  $W^* \leftarrow \text{BUSCALOCAL}(W_0, G, q); \quad f^* \leftarrow f(W^*)$ 
3  $W \leftarrow W^*; \quad \text{sem\_melhora} \leftarrow 0$ 
4 para  $iter \leftarrow 1$  até  $I_{\max}$  faça
5   se  $\text{sem\_melhora} \geq N_{\text{stag}}$  então
6      $W_0 \leftarrow \text{CONSTRUIRGULOSO}(G, q)$ 
7      $W \leftarrow \text{BUSCALOCAL}(W_0, G, q)$ 
8      $\text{sem\_melhora} \leftarrow 0$ 
9    $k \leftarrow 1$ 
10  enquanto  $k \leq K_{\max}$  faça
11     $W' \leftarrow \text{AGITAR}(W, \mathcal{N}_k, G, q)$ 
12     $W'' \leftarrow \text{BUSCALOCAL}(W', G, q)$ 
13    se  $f(W'') < f(W)$  então
14       $W \leftarrow W''$ 
15      se  $f(W) < f^*$  então
16         $f^* \leftarrow f(W)$ 
17         $W^* \leftarrow W$ 
18         $\text{sem\_melhora} \leftarrow 0$ 
19       $k \leftarrow 1$ 
20     $k \leftarrow k + 1$ 
21     $\text{sem\_melhora} \leftarrow \text{sem\_melhora} + 1$ 
22    se  $f^* = 0$  então interrompe
23 retorna  $f^*, W^*$ 

```

Algoritmo 2: BUSCALOCAL — Descida de Primeiro Aprimoramento por Arco Único

Entrada: Passeio $W = \langle v_0, \dots, v_{|W|-1} \rangle$, grafo $G(V, E, \sigma)$, consulta q .
Saída: Passeio W localmente ótimo.

```

1  $\text{melhorou} \leftarrow \text{verdadeiro}$ 
2 enquanto  $\text{melhorou}$  faça
3    $\text{melhorou} \leftarrow \text{falso}$ 
4   para  $i \leftarrow 0$  até  $|W| - 1$  faça
5     se  $i = 0$  então
6        $\mathcal{C} \leftarrow \{v \in V : v \neq v_0, \sigma(v) = q[0]\}$ 
7      $\mathcal{C} \leftarrow \{w \in \delta^+(v_{i-1}) : w \neq v_i, \sigma(w) = q[i]\}$ 
8     para cada  $w \in \mathcal{C}$  faça
9        $W_{\text{teste}} \leftarrow \langle v_0, \dots, v_{i-1}, w \rangle \circ \text{ESTENDERGULOSO}(w, i+1, G, q)$ 
10      se  $f(W_{\text{teste}}) < f(W)$  então
11         $W \leftarrow W_{\text{teste}}$ 
12         $\text{melhorou} \leftarrow \text{verdadeiro}$ 
13        interrompe (reinicia varredura)
14      se  $\text{melhorou}$  então interrompe
15 retorna  $W$ 

```

Algoritmo 3: AGITAR — Perturbação por Estrutura de Vizinhança

Entrada: Passeio W , índice de vizinhança k , grafo $G(V, E, \sigma)$, consulta q .

Saída: Passeio perturbado W' .

```

1 se  $k = 1$  (substituição de arco simples) então
2   | Sorteie  $i$  uniformemente de  $\{1, \dots, |W| - 1\}$ 
3   | Sorteie  $u$  uniformemente de  $\delta^+(v_{i-1}) \setminus \{v_i\}$  (ou de  $\delta^+(v_{i-1})$  se singleton)
4   |  $W' \leftarrow \langle v_0, \dots, v_{i-1}, u \rangle \circ \text{ESTENDERGULOSO}(u, i+1, G, q)$ 
5 senão se  $k = 2$  (substituição de arco dupla) então
6   | Sorteie  $i < j$  uniformemente de  $\{1, \dots, |W| - 1\}$ 
7   | Sorteie  $u$  uniformemente de  $\delta^+(v_{i-1})$ 
8   | Reconstói gulosamente o segmento  $v_i, \dots, v_j$  a partir de  $u$ , depois aplica
   | ESTENDERGULOSO para  $v_{j+1}, \dots, v_{m-1}$ 
9   |  $W' \leftarrow$  passeio resultante
10 senão se  $k = 3$  (reinscrição de segmento) então
11   | Sorteie  $i$  de  $\{0, \dots, |W| - 1\}$  e  $L$  de  $\{2, \dots, \lfloor m/3 \rfloor\}$  uniformemente
12   | Se  $i > 0$ : sorteie  $u \in \delta^+(v_{i-1})$ ; senão: sorteie  $u \in V$  uniformemente
13   |  $W' \leftarrow \langle v_0, \dots, v_{i-1}, u \rangle \circ \text{ESTENDERGULOSO}(u, i+1, G, q)$ 
14 senão
15   | (substituições simultâneas múltiplas de arco —  $\mathcal{N}_4$ )
16   |  $k_{\text{pert}} \leftarrow \min(k, |W| - 1)$ 
17   | Sorteie  $k_{\text{pert}}$  posições distintas de  $\{1, \dots, |W| - 1\}$ , embaralha-as em  $p_1, \dots, p_{k_{\text{pert}}}$ 
18   |  $W' \leftarrow W$ 
19   para  $t \leftarrow 1$  até  $k_{\text{pert}}$  faça
20     |  $i \leftarrow p_t$ 
21     | Sorteie  $u$  uniformemente de  $\delta^+(W'[i - 1])$ 
22     | Trunca  $W'$  ao comprimento  $i$ , acrescenta  $u$  e aplica ESTENDERGULOSO
23 retorna  $W'$ 

```

5. Resultados Computacionais

Esta seção apresenta um comparativo do desempenho computacional entre o modelo de Programação Linear Inteira, resolvido pelo Gurobi 13.0 com limite de tempo de 600 s, e a metaheurística VNS. Desenvolvidas em C++, ambas as abordagens foram testadas em 360 grafos aleatórios baseados no modelo de [Erdős e Rényi, 1959], com variações em $|V| \in \{60, 80, 100, 120\}$, $d \in \{10\%, \dots, 60\%\}$ e $q \in \{10\%, \dots, 90\%\}$. Os parâmetros da VNS foram calibrados experimentalmente: para $q \leq 30\%$, $k_{\text{max}} = 8$ e $N_{\text{stag}} = 10\,000$; para $q \geq 50\%$, $k_{\text{max}} = 12$ e $N_{\text{stag}} = 25\,000$. Os experimentos foram realizados em um Intel Core i5-9300H a 2,40 GHz com 16 GB de RAM.

O limitante dual ao final da execução é denominado *limitante inferior* (LI) e a melhor solução primal é o *limitante superior* (LS). O gap do PLI é $\text{gap}_{\text{PLI}} = (LS - LI)/LS \times 100\%$. Para o VNS, definem-se $\text{gap}_{\text{LI}} = (f_{\text{VNS}} - LI)/LI \times 100\%$ (quando $LI > 0$) e $\text{gap}_{\text{LS}} = (f_{\text{VNS}} - LS)/LS \times 100\%$ (quando $LS > 0$).

A Tabela 1 sintetiza os resultados gerais. O PLI provou otimalidade em 284 instâncias (78.9%); as 76 restantes (21.1 %) esgotaram o limite de tempo, cenário em que o VNS atua como alternativa principal. A metaheurística encontrou alinhamentos perfeitos ($f = 0$) em 10.8 % dos casos e igualou a solução ótima em 31.7 %. O gap médio relativo ao LI atinge 135.67 %, valor

inflado pela assimetria da métrica quando o LI é próximo de zero ($LI = 1$, $f_{VNS} = 2$ já gera 100 %); as análises seguintes complementam as medidas percentuais com os valores absolutos.

Tabela 1: Resumo geral dos resultados computacionais sobre as 360 instâncias.

Algoritmo	Indicador	Valor
PLI	Instâncias resolvidas até a otimalidade	284 (78,9%)
	Instâncias com limite de tempo atingido	76 (21,1%)
	Gap nas instâncias com Tempo Limite atingido (TL) (Média / Máx.)	46,31% / 96,20%
VNS	Alinhamentos perfeitos ($f = 0$)	39 (10,8%)
	Igualou o LS do PLI ($f_{VNS} = LS$)	114 (31,7%)
	Gap médio relativo ao PLI (LS / LI)	69,60% / 135,67%

O comprimento relativo da consulta é o fator de maior impacto. A Tabela 2 e a Figura 3 apresentam os resultados agregados por nível de q . Em $q = 10\%$, o PLI resolve todas as instâncias em menos de 1 s e o VNS obtém gap médio de 8.67 %. A taxa de otimalidade cai com q : 94.4 %, 80.6 %, 65.3 % e 54.2 % para $q \in \{30\%, 50\%, 70\%, 90\%\}$. Em $q = 90\%$, o gap médio do PLI nas instâncias com TL atinge 56.62 % (máx. 96.20 %), evidenciando limitantes inferiores fracos em grafos esparsos. O custo médio do VNS cresce de 1.44 a 16.60, com o gap VNS–LS variando de 8.67 % a 129.28 %; em $q = 90\%$ o VNS supera o LS do PLI em 7 casos.

Tabela 2: Resultados agregados por tamanho relativo de consulta (q). n_{inst} : número de instâncias; %OPT: percentual resolvido pelo PLI até a otimalidade; $\overline{gap}_{PLI}$: gap médio do PLI nas instâncias com TL; $\bar{f}_{VNS}$: custo médio do VNS; $\overline{gap}_{LI}$ e $\overline{gap}_{LS}$: gaps médios do VNS em relação ao LI e ao LS; $\bar{t}_{PLI}$ e $\bar{t}_{VNS}$: tempos médios de execução (s).

q	n_{inst}	%OPT	$\overline{gap}_{PLI}(\%)$	$\bar{f}_{VNS}$	$\overline{gap}_{LI}(\%)$	$\overline{gap}_{LS}(\%)$	$\bar{t}_{PLI}(s)$	$\bar{t}_{VNS}(s)$
10%	72	100,0	00,00	1,44	8,67	8,67	0,1	4,0
30%	72	94,4	22,36	4,67	31,99	30,04	45,9	19,0
50%	72	80,6	31,44	8,67	89,90	73,83	154,7	124,3
70%	72	65,3	44,86	12,01	170,71	85,66	246,7	401,3
90%	72	54,2	56,62	16,60	330,95	129,28	322,8	492,6

A densidade de arestas afeta diretamente a variedade de caminhadas disponíveis para o alinhamento. A Tabela 3 e as Tabelas 4–5 apresentam os resultados por d e a interação $d \times q$. O efeito sobre o PLI é monotônico: em $d = 10\%$, apenas 38.3 % das instâncias são resolvidas até a otimalidade (gap médio de 53.43 %, tempo médio de 384.3 s); a taxa sobe para 81.7 %, 96.7 % e 100 % em $d \in \{30\%, 40\%, 50\%\}$. O VNS exibe comportamento oposto: o custo médio decresce de 19.37 ($d = 10\%$) a 2.97 ($d = 60\%$), confirmando que grafos mais densos oferecem caminhos de maior qualidade. O tempo do VNS cresce com a densidade, atingindo o pico em 333.5 s em $d = 60\%$, pois instâncias densas com consultas longas demandam a execução completa das vizinhanças. O gap VNS–LS é reduzido em $d = 10\%$ (15.25 %) porque o PLI raramente produz soluções primais fortes nesses grafos; em grafos densos, onde $LS = LI$, o gap relativo se estabiliza entre 70 % e 110 %.

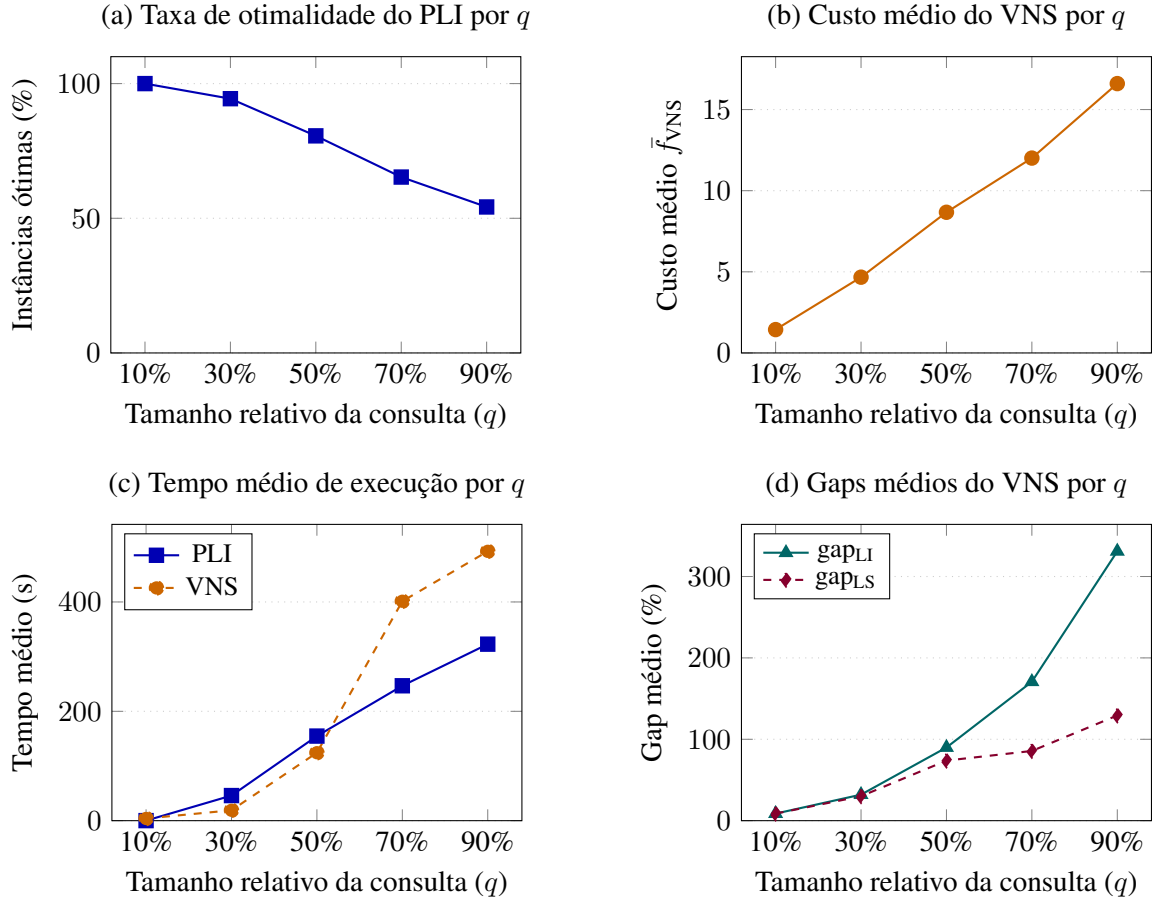


Figura 3: Tendências dos principais indicadores em função do tamanho relativo de consulta (q). O painel (c) inclui instâncias que atingiram o limite de 600 s para o PLI. O gap do PLI é omitido em $q = 10\%$ pois todas as instâncias foram resolvidas até a otimalidade.

Tabela 3: Resultados agregados por densidade de arestas (d).

d	n_{inst}	%OPT	$\overline{\text{gap}}_{PLI}(\%)$	$\bar{f}_{VNS}$	$\overline{\text{gap}}_{PLI}(\%)$	$\overline{\text{gap}}_{LS}(\%)$	$\bar{t}_{PLI}(s)$	$\bar{t}_{VNS}(s)$
10%	60	38,3	53,43	19,37	214,44	15,25	384,3	53,9
20%	60	56,7	41,11	12,07	154,73	61,17	295,9	114,0
30%	60	81,7	35,45	8,22	138,66	88,12	158,6	161,9
40%	60	96,7	41,67	5,68	90,74	83,02	57,6	261,8
50%	60	100,0	—	3,77	109,67	109,67	20,9	324,3
60%	60	100,0	—	2,97	74,64	74,64	6,8	333,5

Tabela 4: Percentual de instâncias resolvidas até a otimalidade pelo PLI.

Dens. (d)	Consulta (q)					Dens. (d)	Consulta (q)				
	10%	30%	50%	70%	90%		10%	30%	50%	70%	90%
10%	100,0	66,7	25,0	0,0	0,0	10%	3,25	9,67	18,50	28,42	37,00
20%	100,0	100,0	58,3	25,0	0,0	20%	2,17	6,67	11,92	16,08	23,50
30%	100,0	100,0	100,0	66,7	41,7	30%	1,17	4,58	8,33	11,92	15,08
40%	100,0	100,0	100,0	100,0	83,3	40%	1,33	3,50	4,75	7,42	11,42
50%	100,0	100,0	100,0	100,0	100,0	50%	0,42	1,67	5,17	4,17	7,42
60%	100,0	100,0	100,0	100,0	100,0	60%	0,33	1,92	3,33	4,08	5,17

Tabela 5: Custo médio do VNS por combinação de d e q .

A Tabela 4 confirma que o PLI não provou otimalidade em nenhuma instância com $d \leq 20\%$ e $q \geq 70\%$, enquanto qualquer combinação com $d \geq 50\%$ é resolvida até a otimalidade em todos os casos. A Tabela 5 reforça que o custo médio do VNS decresce com d e cresce com q , com pior caso em $d = 10\%$, $q = 90\%$ ($\bar{f} = 37,00$) e melhor em $d = 60\%$, $q = 10\%$ ($\bar{f} = 0,33$); o ganho marginal de densidade é decrescente, com a redução de $d = 10\%$ para 20% poupando 13.5 incompatibilidades em $q = 90\%$ contra apenas 2.25 da redução de 50% para 60% .

O efeito de $|V|$ é apresentado na Tabela 6. O PLI torna-se progressivamente mais difícil: a taxa de otimalidade cai de 88.9 % ($|V| = 60$) a 70.0 % ($|V| = 120$), e o tempo médio quase triplica (81.7 s a 214.2 s). O VNS exibe comportamento contraintuitivo: o custo médio *diminui* de 9.09 a 8.39 porque grafos maiores, mantidas as proporções de q e d , oferecem mais caminhos viáveis de alta qualidade. O tempo do VNS cresce de 90.1 s a 346.7 s, com pico em $|V| = 120$, $q = 90\%$ (762.4 s).

Tabela 6: Resultados agregados por número de vértices ($|V|$).

$ V $	n_{inst}	%OPT	$\overline{\text{gap}}_{\text{PLI}}(\%)$	$\bar{f}_{\text{VNS}}$	$\overline{\text{gap}}_{\text{LI}}(\%)$	$\overline{\text{gap}}_{\text{LS}}(\%)$	$\bar{t}_{\text{PLI}}(\text{s})$	$\bar{t}_{\text{VNS}}(\text{s})$
60	90	88,9	32,89	9,09	47,26	39,15	81,7	90,1
80	90	78,9	34,00	8,93	82,30	62,50	146,4	155,6
100	90	77,8	48,33	8,30	128,54	66,35	173,8	240,6
120	90	70,0	58,44	8,39	324,36	121,37	214,2	346,7

As instâncias mais desafiadoras são as com $d = 10\%$ e $q = 90\%$: o PLI não provou otimalidade em nenhum dos 12 casos, com gaps residuais entre 46.2 % e 96.2 % e LS variando de 26 a 81, distante do $LI \in [3, 14]$. O VNS produziu soluções entre 27 e 52 incompatibilidades, superando o LS do PLI em 7 dos 12 casos. Para a configuração mais extrema ($|V| = 120$, $d = 10\%$, $q = 90\%$), o PLI retornou $LS \in \{70, 79, 81\}$ contra $LI \in \{3, 3, 4\}$, enquanto o VNS produziu custo $\in \{45, 46, 52\}$, reduzindo o melhor valor primal em mais de 35 %, evidenciando sua robustez nos cenários em que a abordagem exata é insuficiente.

6. Conclusão

Este trabalho apresentou uma metaheurística Variable Neighborhood Search para o Problema de Alinhamento de Sequência em Grafos com Modificação de Rótulos de Vértices. Para viabilizar a avaliação rigorosa do VNS, foi empregada uma formulação PLI como referência: nas

instâncias tratáveis, ela forneceu soluções ótimas para comparação; nas demais, seus limitantes inferiores e superiores delimitaram o espaço de qualidade dentro do qual os resultados da metaheurística foram interpretados.

Os resultados experimentais demonstram a eficácia do VNS ao longo do conjunto de testes: a metaheurística obteve alinhamento perfeito ($f = 0$) em 10.8 % das instâncias e igualou a solução ótima em 31.7 % dos casos. Os resultados mais expressivos concentram-se nas instâncias com consultas longas ($q = 90\%$): dentre as 33 instâncias em que o solver esgotou o limite de 600 s, o VNS melhorou o melhor limitante superior conhecido em 7 delas. No cenário mais extremo ($|V| = 120$, $d = 10\%$, $q = 90\%$), o VNS reduziu o custo primal disponível em mais de 35 %, evidenciando sua utilidade como alternativa robusta quando a abordagem exata se mostra insuficiente.

Para trabalhos futuros, duas extensões naturais apresentam direções promissoras: a incorporação de uma busca local mais agressiva entre as iterações da VNS e o uso de estruturas de vizinhança adaptativas, desenvolvidas para priorizar regiões do espaço de soluções com maior potencial de melhoria.

Referências

- Amir, A., Lewenstein, M., e Lewenstein, N. (2000). Pattern matching in hypertext. *Journal of Algorithms*, 35(1):82–99.
- Erdős, P. e Rényi, A. (1959). On random graphs i. *Publicationes Mathematicae Debrecen*, 6: 290–297.
- Hansen, P. e Mladenović, N. (2001). Variable neighborhood search: Principles and applications. *European Journal of Operational Research*, 130(3):449–467.
- Hansen, P., Mladenović, N., e Moreno Pérez, J. A. (2010). Variable neighbourhood search: Methods and applications. *Annals of Operations Research*, 175(1):367–407.
- Ivanov, P., Bichsel, B., Mustafa, H., Kahles, A., Rätsch, G., e Vechev, M. (2020). Astarix: Fast and optimal sequence-to-graph alignment. In *International Conference on Research in Computational Molecular Biology*, p. 104–119. Springer.
- Jain, C., Misra, S., Zhang, H., Dilthey, A., e Aluru, S. (2019). Accelerating sequence alignment to graphs. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, p. 451–461. IEEE.
- Jain, C., Zhang, H., Gao, Y., e Aluru, S. (2020). On the complexity of sequence-to-graph alignment. *Journal of Computational Biology*, 27(4):640–654.
- Jiménez-Blanco, A., López-Villellas, L., Moure, J. C., Moreto, M., e Marco-Sola, S. (2026). The-seus: Fast and optimal affine-gap sequence-to-graph alignment. *bioRxiv*, p. 2026–02.
- Rautiainen, M. e Marschall, T. (2020). Graphaligner: rapid and versatile sequence-to-graph alignment. *Genome biology*, 21(1):253.
- Sansford, H., Richardson, N., Maretic, H. P., e Saada, J. N. (2024). Grapheval: A knowledge-graph based llm hallucination evaluation framework. In *KiL@KDD*. URL <https://api.semanticscholar.org/CorpusID:271197177>.
- Vaser, R., Sović, I., Nagarajan, N., e Šikić, M. (2017). Fast and accurate de novo genome assembly from long uncorrected reads. *Genome Research*, 27(5):737–746.